Introduction to messaging

Messaging

- Communication of programs
- Message passing: asynchronous communication
- Loose binding of components by using channels

Messaging is a way of communication between computer programs over a network. The most important aspect of it is the loose binding of communicating components. This means the sender and a receiver of a piece of information (i.e. a message) is not connected directly, and they may do not know anything about each other at all. Architecturally, it is possible by using a so-called `messaging provider` between the components; they need to know just how to communicate with this provider, and not with the other components. As a result of this architecture we can easily connect various systems which are written in different languages, and using diverse technologies. Using messaging providers also enables asynchronous communication: a receiver of a message is not necessarily needed to be available and actively receive the message at the same time when it was sent.

Comparison with remote procedure call

- RPC like protocols (e.g. RMI, CORBA, DCOM, SOAP)
  - Synchronous communication
  - Relies on a common language
  - Strong coupling
- Email, instant messaging
  - Between humans
  - Message passing between computer and human

Compared to communication based on remote procedure calls (RPC), some benefits of messaging are obvious. In most of the cases, RPC works synchronously. When a call is made on a remote procedure, the server and the client must be running and available on the network during the whole time of the call, at the same time. Because the server and the client is connected directly, they have to be configured to each other; at least some details about the connection in the client are needed. Moreover, the direct connection requires the peers to use a common communication protocol. These two properties of RPC are making it a relatively strongly coupling technique.

As we have seen before, in the case of messaging, the communication is done through a messaging provider. The main task of a provider is to ensure the delivery of the messages through its channels. However, during the delivery it can use even different

protocols, and this way eliminate the incompatibility of protocols used by the components on the sender and receiver side. The peers are only needed to know how to communicate to the provider, and which channel should they use. Providers can also be used to make the communication more robust and reliable, as they can be able to handle transactions, persist messages, log communication, and offer other guarantees on message delivery.

Components to integrate

- Enterprise Resource Planning
- Customer Relationship Management
- Data warehouse
- Management Information System, data mining
- Knowledge or workflow management
- Groupware
- `B2B` and `B2C`
- Electronic Document Interchange

Messaging is a widely used technology in the industry. Its most common usage field is system integration, where systems from possibly different vendors are connected together to build a more complex system. This is needed to automate processes which are working with data from different departments or even from different companies.

Java Message Service

Overview

`JMS` Overview

- `Java Message Service` (`JMS`)
  - Standard for cooperation
  - Part of Enterprise Java specification
  - Common programming interface, `API`
- Coupling the components, creating communication between them
- Message-Oriented Middleware (`MOM`)

`Java Message Service` is a messaging specification for components written in the Java programming language. It defines a programming interface, which describes how Java programs should communicate with a `JMS`-capable messaging provider.

JMS Providers

Proprietary

- BEA
- IBM
- iPlanet (Sun)
- Oracle
- TIBCO
- SeeBeyond
- Software AG.

Open source

- Apache ActiveMQ
- JBoss HornetQ
- JORAM
- OpenJMS
- OpenMQ

In the last decades, every important software company developed its own messaging solution. Currently, the reference implementation of JMS - which is an evidence that it can be implemented - is OpenMQ. Because these applications provide services for others which are using them, the term middleware is often used. By providing messaging services for distributed systems, the term Message-Oriented Middleware is also became widespread. Traditionally, in this context, the component which provides the messaging service is known also known as the ``message broker''.

Messaging models (JMS Domains)

- Point-to-point (PTP or P2P) model
  - Traditional model, uses a message queue
  - Messages can be appended into and removed from the queue
  - Messages are passed between two endpoints (programs, components)
- Publish-subscribe (Pub/Sub) model
  - Receivers are subscribing to topics
  - Senders can publish messages on these topics
  - A topic broadcasts the messages to all of its subscribers
  - Increasingly popular

JMS supports two different messaging models. Basically, the model defines how the messaging channel will behave during the communication. JMS generally uses the term ``destination'' to a messaging channel, and the term ``domain'' to a messaging model.

The point-to-point messaging model enables us to send messages between two endpoints. In this case, the messaging channel is a traditional message queue. This almost works like an ordinary queue: a sender can append messages to its end, and a receiver can process messages from it on the other end. Multiple senders can use a single queue. However, when a message is successfully processed, it is removed from the queue - so if there are multiple components which are trying to get the message, only one of them will receive it.

Another possible way of communication is by using the publish-subscribe domain. The communication channel is a so-called topic in this model. As in the case of message queues, multiple senders can publish messages to a topic. But the main difference here is, if multiple receivers are subscribed to a single topic, then each one of them will receive every published message - so it will be broadcasted for all the subscribers of the given topic.

When using message queues, the provider stores its messages, so when a receiver goes offline, and stops processing messages, it will get all ``missed'' messages when it become online again, and starts the processing. Messages published to a topic are a little bit different: when a subscriber goes offline and then online again, it will get the missed messages only when it has a special, so-called ``durable subscription''.

Architecture

Core elements of the JMS architecture

- JMS clients
- JMS provider
- Persistent JMS objects (administered objects)
  - Connection factories
  - Message destinations: queues and topics

The clients are encapsulating the business logic of the given application. They are using the messaging provider to send and receive messages through it. If a message provider conforms to the JMS specification, it is guaranteed that all JMS clients can use it - this ensures portability and the ease of integration of messaging software. Of course, JMS-capable providers often support multiple communication protocols or messaging standards, for example including the vendor's own solutions. This further supports connectivity of messaging systems. A messaging provider is often a part of a bigger middleware solution, like an application server.

Generally, a client of a messaging provider needs to be configured with two set of parameters. First, it has to connect to the provider physically. This is achieved by some kind of network connection, which is - in the case of JMS - an instance from a connection factory. The connection factory holds the necessary host and port addresses, etc. to build up the connection. The another set of parameters is describing the messaging channel, the message destination. In JMS, it is a named resource, which encapsulates information about the chosen messaging model, the resource limits, authorization policies, and so on.

These set of parameters are stored as so-called administered objects. They are the part of the system's configuration, created by the administrators usually at the time of application deployment. In the case of the Java platform, named resources like these are usually accessed through the Java Naming and Directory Interface (JNDI) service. Another common way to get these resources is using dependency injection provided generally by application servers.

Versions of the JMS API

- Classic API
  - Domain-dependent (since version 1.0)
  - Domain-independent (since version 1.1)
  - Uses checked exceptions (JMSException)
  - Most of the existing applications are using this API
- Simplified API (since version 2.0)
  - Shorter and easier usage, less initialization
  - Uses unchecked exceptions (JMSRuntimeException)

The first JMS specification was designed and released in the late nineties. After releasing an improved version 1.1 in 2002, which introduced a common API for the different messaging models, the developers left it unchanged for almost a decade. With a major update to version 2.0 in 2013, they created a new, simplified API which helps to reduce the amount of code needed by the client applications.

The major difference between the domain-dependent and independent API is the latter does not make difference between the handling of topics and queues. This means in most of the cases the same application can be used with both messaging models. From version 1.1, this is the preferred way of using the API. The simplifications introduced in version 2.0 will be shown later.

Classic API

Elements of the domain-independent JMS API

- Interfaces in the `javax.jms.*` package
- `ConnectionFactory`, `Connection`
- `Destination`
- `Session`
- `MessageProducer`, `MessageConsumer`
- `Message`
- Example 1 (topic) and Example 2 (queue)

To use the JMS service, first we have to create a connection factory and a message destination. As these are administered objects, the specification does not describe any rules how they should be created or managed - this depends on the actual JMS provider. The common practice is to look up these objects by their name with JNDI. (When using an application server, the version 2.0 of the specification allows automatic, deployment-time creation of these objects from descriptions given as e.g. annotations - more on that feature later.)

From a `ConnectionFactory`, we can create an instance of `Connection`, which represents the physical connection to the JMS provider. This is a thread-safe, heavyweight object, a typical application has only one, or very few instances of this during its lifetime.

`Session` is a lightweight object representing a messaging context. It can be created from a `Connection` instance, and may be used only from a single thread. It has multiple roles in the JMS ecosystem, but from the API user's aspect the most important is that its the factory of message producers and consumers.

As the name suggests the message producers and consumers can be used to send or receive messages to or from a destination - which can be either a message queue or topic. The `Message` interface is the common ancestor of specific message types. Currently there are five type of messages depending on what type of data a message body can contain.

We will examine these interfaces in detail later.

Symmetry of the domain-dependent `JMS API`

- `Queue` and `Topic` is similar
  - `QueueConnection` / `TopicConnection` -> `Connection`
  - `Queue` / `Topic` -> `Destination`
  - `QueueSender` / `TopicPublisher` -> `MessageProducer`
  - `QueueReceiver` / `TopicSubscriber` -> `MessageConsumer`
  - `send(...)` / `publish(...)`
  - etc.
- What about `QueueBrowser`?

  - There is no corresponding interface for topics
  - More about this interface later

As we can see the domain-dependent API were highly redundant. Most of the interfaces were duplicated, with a prefix Queue or Topic indicating the appropriate domain. From version 1.1, it is not recommended to write new software using these interfaces. However, some interfaces like QueueBrowser, which does not have any counterpart in the other model, remained in use.

Connection factory

- Typically preconfigured and available through JNDI
  - javax.naming.Referenceable and java.io.Serializable
  - Contains information about how to reach a JMS provider

```
Context context = new InitialContext();
ConnectionFactory factory =
    (ConnectionFactory)context.lookup(
        "java:global/jms/myConnectionFactory");
```

OR

```
@Resource(lookup="java:app/jms/myConnectionFactory")
private ConnectionFactory factory;
```

Depending on the application's environment, we can obtain a connection factory either by using normal JNDI lookup, or by injecting it as a resource. The factory itself is an administered object, which must be configured before the first usage, e.g. on application deployment.

Destination

- An entity which represents either a queue or topic
  - Configured during application deployment
  - May be created temporarily
- Used by:
  - Message producers to send messages
  - Message consumers to process messages

```
Destination destination =
    (Destination)context.lookup(
        "java:global/jms/myMessageQueueOrTopic");
```

OR

```
@Resource(lookup="java:app/jms/myMessageQueueOrTopic")
private Destination destination;
```

To send or receive messages, we have to specify a destination for our message producers and consumers. Destinations are created and accessed in a similar way as connection factories. We can either use normal JNDI lookup or resource injection in a container. If we do not want to change the type of the destination later, we can use the Queue or Topic domain-specific interfaces to store the reference.

Connection

- Represents a connection to a JMS Provider
- Multi-threaded
- Provides authentication
- Heavy-weight
  - Usually one instance is enough per client
  - It uses resources outside of the Java virtual machine
  - It must be closed after usage (java.lang.AutoCloseable)
- Must be started to receive incoming messages

```
... // factory setup
Connection connection = factory.createConnection();
... // setup of message consumers
connection.start();
...
connection.close();
```

A connection to the provider can be obtained by creating it from the factory. For authentication and authorization purposes, there is an optional creation method, which accepts a user name and a password as a string. If these are omitted, the connection will use a default identity. After the proper setup of message consumers is done, the connection must be started to receive the messages. The message delivery can be stopped and restarted multiple times. Message producers can send messages even when the connection is stopped; its state affects only the processing of incoming messages. As a connection is a heavy-weight object, it has to be closed after usage. It is an auto-closeable object, so it can be used with a try-with-resources construction.

Session

- Single threaded
- To produce and consume
- Serializes message flow
- Specifies message acknowledgment mode
- To obtain transactions (depends on the instantiation)

```
Session session = connection.createSession();
```

OR

```
// select one of the valid session modes, see next slides
int sessionMode = ...;
Session session = connection.createSession(sessionMode);
```

A session is a single-threaded context to encapsulate message producing and consuming processes. It specifies the message acknowledgment mode, and its behavior and relation to transactions. Messages sent or received in a single session are serialized: the processing will have a deterministic order regarding to a single session. If a session is rolled back, all messages ready for send will be destroyed, and all received messages will be marked as unread - so they may be redelivered later.

Session mode

- Specifies message acknowledgment and transaction mode
- Constant values on the `Session` interface
  - `SESSION_TRANSACTED` (`session.commit()`, `session.rollback()`)
  - `CLIENT_ACKNOWLEDGE` (`message.acknowledge()`)
  - `AUTO_ACKNOWLEDGE`
  - `DUPS_OK_ACKNOWLEDGE`
- Before `JMS` 2.0, it was expressed with two parameters on creation

```
Session createSession(boolean transacted, int acknowledgeMode);
```

The accepted session modes are varying depending on the context of application. When running in an enterprise web or `EJB` container, and a `JTA` transaction is in progress, the session mode will be ignored, and the session will join to the transaction. If there is no active transaction, `AUTO_ACKNOWLEDGE` and `DUPS_OK_ACKNOWLEDGE` may be used. In the first case, the messages will be automatically acknowledged if the processing is successful (i.e. no `JMS` exception thrown during the processing). The

second mode is better for performance: if receiving a message twice is not a problem, then it can do the same as `AUTO_ACKNOWLEDGE`, but the acknowledgments can be delayed some times. If the client is running as a standard edition application or in an enterprise application client container, all four modes are possible. In the `SESSION_TRANSACTED` mode it uses a local transaction, which can be committed or rolled back by method calls. When the mode is `CLIENT_ACKNOWLEDGE`, then a call of method `acknowledge()` on any message instance will acknowledge all received messages in the session. The other two modes will work as the same as above.

Transactions

- Session: commit and rollback
  - If the session is created such a way
  - Context of the application is also matters
  - The messages are delivered one-by-one
- After commit, all the sent messages are delivered
- After rollback, all the read messages are marked unread
- Example 5 (sender), Example 6 (receiver)
- Distributed transactions
  - `JTA` (maybe `JTS`)
  - `XASession`, `XAConnection`

Acknowledgment

- Lost message: should we resend?
- Not necessary if it is acknowledged

The acknowledgment mode controls how the provider is notified about the success of processing a message. If a message is acknowledged in the provider, it will not try to redeliver it.

Session mode - `CLIENT_ACKNOWLEDGE`

- `message.acknowledge()`
- Acknowledges all received messages in the current session
  - Message group
- Example 7

Session mode - `DUPS_OK_ACKNOWLEDGE`

- If multiple delivery is not a problem
  - Redelivers twice only if the `JMS` provider fails
- Better performance than with automatic acknowledgment

Session mode - `SESSION_TRANSACTED`

- If the session is transaction aware
- Messages are acknowledged automatically after committing
- Also can be rolled back
  - Prepared messages will not be sent
  - Marks every received message as unread

Message producer

- Sends messages into a queue, or publishes them to a topic
- Can be used to specify default properties of messages
  - Priority
  - Delivery mode
  - Time to live
  - More on these properties later
- Usually created for a single, fixed destination (but not always)

```
MessageProducer producer = session.createProducer(destination);
Message message = ...; // create a message, see later
producer.send(message);
```

Message producers are able to send messages to a given destination. This destination can be either a topic or queue. In most of the cases, the destination is fixed, and provided at the creation of a message producer. If the provided reference is `null`, the destination must be specified at every send as a parameter. The `send` method has many overloaded versions, thus it is possible to specify the destination, delivery mode, priority, and time to live values for each message individually. However, a great benefit of message producers that we can set these properties as defaults on them, so the calls of `send` method can be simpler. By default, this method makes a synchronous call to the provider, i.e. it will block the current thread until it receives an acknowledgment that the provider successfully received the message. `JMS` 2.0 also provides an overloaded version with a `CompletionListener` parameter, which provides asynchronous semantics.

Structure of a message

- Main parts:
  - Headers
  - Properties
  - Body or payload
- Different message types for different kinds of payloads
- All inherited from a unified `Message` interface

Every `JMS` message has three main parts. First, it has a set of headers. Each header contain meta-data for the provider, in the form of a name-value pair. They are needed to control the delivery of the message. `JMS` 2.0 defines eleven mandatory headers. Some of them are set by the client, and some of them by the provider.

The second part of a message contains additional, application specific meta- information, called properties. Basically, these are custom header fields with a name and a value. They can be used e.g. to filter messages by some criteria, or to control a vendor-specific extension of the messaging provider.

Of course, the third, last and the most important part of a message is its body, the payload data itself. `JMS` 2.0 provides five different message interfaces, and each one is able to set and extract a different kind of payload.

Operations on messages

- Using the common `Message` interface
- Querying and setting headers and properties
- Reading and writing the body
  - Through different derived interfaces
- Sending acknowledgment
  - In case of session mode `CLIENT_ACKNOWLEDGE` or `DUPS_OK_ACKNOWLEDGE`
- The message object is reusable, can be sent multiple times

The `Message` interface contains getter and setter methods to adjust some of the message headers, and manage properties. It also provides an `acknowledge` method, which is used to send an acknowledgment about the processed messages to the provider in case of session modes with manual message acknowledgment. To ensure type-safe handling of the different message bodies, `JMS` provides a derived interface for each message body type.

Text message

- Contains a `java.lang.String`
- Also useful for `XML` or `JSON` data

```
TextMessage message = session.createTextMessage("message body");
```

OR

```
TextMessage message = session.createTextMessage();
message.setText("message body");
...
String text = message.getText();
```

Text messages can store a single Java `String`. The default value is a `null` reference.

Object message

- Holds a single Serializable Java object (Example 9)

```
ObjectMessage message =
    session.createObjectMessage(serializableObject);
```

OR

```
ObjectMessage message = session.createObjectMessage();
message.setObject(serializableObject);
...
Serializable object = message.getObject();
```

An object message can hold a single Java object, which is from a type that implements the `java.io.Serializable` interface. The default value is `null`. After an object is extracted from a message, it should be converted to the appropriate type.

Stream message

- Contains a sequence of primitive values
  - Boolean values
  - Numbers (byte, short, int, long, float, double)
  - Characters, strings

- Order of writing and reading of elements must be the same

```
StreamMessage message = session.createStreamMessage();
// can be used like an ordinary Java data input or output stream
message.writeInt(42);
message.writeString("Hello JMS!");
...
int theAnswerToTheUniverse = message.readInt();
String greeting = message.readString();
```

A stream message contains an ordered sequence of Java primitive types (including `String`). It provides methods to read and write its body as an input or output stream.

Map message

- Contains an associative map of primitive values
- Uses `String` as the key type
- Provides random access of data

```
MapMessage message = session.createMapMessage();
message.setString("greeting", "Hello JMS!");
message.setInt("answer", 42);
...
int theAnswerToTheUniverse = message.getInt("answer");
String greeting = (String)message.getObject("greeting");
```

Map messages are able to hold the same set of types, but each value has an associated name, so the elements are unordered, and their access can be random. The key names can be enumerated by calling method `getMapNames()`. One can also check the existence of a key with `itemExists(key)`. If the type of a value corresponding to a given key is unknown, the `getObject(key)` getter method can be used to extract that value.

BytesMessage

- Holds a raw uninterpreted octet-stream
- Primarily for interoperability

```
BytesMessage message = session.createBytesMessage();
// resembles to the stream message
// but raw bytes are written and read without type markers
// (or any extra information)
message.writeInt(42);
```

```
message.writeUTF("Hello JMS!");
...
int theAnswerToTheUniverse = message.readInt();
String greeting = message.readUTF();
```

The body of a byte message consists of raw bytes, and it can be useful e.g. when one have to cooperate with other messaging systems than JMS, and wants to support their native message format.

Message headers

- They control the message delivery
  - By providing meta-information for the provider
- Every header has a name and a value
- All of them are mandatory
- Information about:
  - Message identifier and destination
  - Delivery mode and time
  - Message expiration and priority
  - Type of the payload

JMS 2.0 specification defines eleven mandatory header fields. Each of them has a name prefixed with JMS. Most of the values are set by the provider through the send method of a message producer object. The Message interface provides getter methods to access values of the predefined headers.

Each message has an identifier. This value is used to uniquely identify the message in the provider. It also can be used to link messages through a correlation id. Destination contains a reference to the queue or topic where the message was sent. Because message delivery can be delayed, and the delivery itself takes some time, the provider writes the delivery time into a header field. Messages can also have expiration time and priority. To help clients to process the message, each of them contains the type of its payload.

Delivery mode

- Controls message persistence
- Can be specified when sending a message
  - `DeliveryMode.PERSISTENT`
    - Message saved and restored after a provider failure
  - `DeliveryMode.NON_PERSISTENT`

        * Not saved, cannot be restored
        * Performance benefits

```
producer.setDeliveryMode(DeliveryMode.NON_PERSISTENT);
producer.send(message);
```

OR

```
producer.send(message, DeliveryMode.NON_PERSISTENT, ..., ...);
...
int deliveryMode = message.getJMSDeliveryMode();
```

The delivery mode can be set for a whole message producer, or individually for each message with the send method. As a standard header, its current value can be acquired with a getter from a received message.

Expiration

- TTL (time-to-live)
- Specified in milliseconds
- JMS Provider tries not to deliver expired messages
- Default value is zero (no expiration)
- In the transactional case, the time of the commit is not relevant
- Example 10

```
producer.setTimeToLive(42 * 1000); // 42 seconds
producer.send(message);
```

OR

```
producer.send(message, deliveryMode, ..., 42 * 1000);
```

Priority

- Every message has a priority from 0 to 9
- Higher priority number means more important message
  - Normal priority from 0 to 4
  - Expedited priority from 5 to 9
  - Default value is 4
- The JMS provider tries to deliver high priority messages first

  - Message queues are priority queues
  - But the ordering by priority is not needed to be guaranteed

```
producer.setPriority(7);
producer.send(message);
```

OR

```
producer.send(message, deliveryMode, 7, timeToLive);
```

Despite its name, a message queue is not entirely a FIFO structure. If multiple senders are appending messages at the same time concurrently, the order of the messages will be non-deterministic. When a message has a higher priority, it should be removed from the queue earlier, than the messages with lower priority. However, the JMS specification does not strictly requires this from a provider.

Properties

- JMS defined properties
  - Names starting with JMSX are reserved
  - Optional headers
  - Redelivery count, transaction identifiers, ...
- Application-defined properties
  - For custom meta-data
  - Primitive values only, separate getter and setter methods by type
- They can be used in message selectors
  - Receive messages only which are matching a criteria

```
message.setIntProperty("answer", 42);
...
int theAnswerToTheUniverse = message.getIntProperty("answer");
```

Properties have two main roles. First, because all JMS headers are mandatory, the only way to define optional headers is to specify them as properties. Second, applications can use them to attach additional information to a message without altering its payload. Another benefit of properties that they can be used to filter messages by a custom criteria.

Message consumer

- Receives messages from a message queue or topic
- Only to receive messages
- Connection must be in started state
- Blocking and non-blocking receive operations (Example 8)
- A message selector can be given at construction (see later)

```
MessageConsumer consumer = session.createConsumer(destination);
// blocking receive, waits for a message
Message message = consumer.receive();
```

OR

```
// non-blocking receive, returns immediately
Message message = consumer.receiveNoWait();
```

Message consumers are used to receive messages from a single destination. It offers synchronous message processing with the `receive` and `receiveNoWait` methods. Each method will return `null` if the message consumer is closed. A call of `receiveNoWait` will also return `null` when there is no message to receive on the destination queue or topic at the time of the call. Because calling the argument-less `receive` can block theoretically forever, an overloaded version also exists, which can be parametrized with a timeout expressed in milliseconds. It also returns `null` when the specified timeout has elapsed.

Synchronous and asynchronous processing

- Synchronous receiving method
  - Variations of `receive`
  - The client is either blocked or polling
- Asynchronous processing
  - Uses a callback object, like event handling
  - Uses the `MessageListener` interface
  - Non-reentrant, a session uses a single thread and serializes processing
  - Example 3 and Example 4

```
consumer.setMessageListener(new MessageListener(){
    @Override
    void onMessage(Message message) {
        ... // process message
    }
});
```

With asynchronous message processing one can use a callback object to get notified when a message arrives to a destination. Because a single session serializes the processing of its messages, a message listener which is used by a single session will never be called while it is currently running. The connection must be in started state to get the listener called on incoming messages.

Durable subscription

- An ordinary subscriber misses messages when it is offline
- For durable subscribers, the provider collects the messages
- When it become online again, the missed messages will be delivered
- Subscription can be destroyed by unsubscribing
- Conditions:
  - Same connection
  - Same destination and name
  - Same message selector
- Example 15

```
session.createDurableSubscriber(topic, "subscription name");
...
// do not collect messages anymore
session.unsubscribe("subscription name");
```

When a consumer of a topic goes offline, it misses the messages until it become online again. With a durable subscription we can inform the provider to persist those messages, and deliver them to the consumer when it becomes online. To identify the subscription, a name must be provided. A consumer can use an existing durable subscription only when its parameters are matching those that are used to create the subscription. A durable subscription can be destroyed by unsubscribing with its name. This enables the provider to remove persisted messages and stop collecting them for the given subscription.

Shared subscription

- Introduced in JMS 2.0
- To share the message processing of a single subscription
- A message will be processed only by one of the consumers sharing a subscription
- Durable shared subscriptions can also be made

```
session.createSharedConsumer(topic, "subscription name");
```

OR

```
session.createSharedDurableConsumer(topic, "subscription name");
```

Shared subscriptions can be used to distribute the work of processing the messages of a single subscription. For every message, only one of the shared subscribers will receive it for a single subscription. Using shared consumers for a durable subscription is also possible.

Message selector

- Filtering messages from a destination
- Headers and properties can be used
- Subset of the SQL92 conditional expression syntax
  - Literals: boolean, numeric, string
  - Identifiers to refer header and property names
  - Comparison operators: =, >, >=, <, <=, <>
  - Logic operators: NOT, AND, OR
  - Arithmetic operators: *, /, +, -
  - Other expressions: `BETWEEN`, `LIKE`, `IN`, `IS`
  - Uses three-valued logic: `TRUE`, `FALSE` and `NULL`
- Example 14

```
MessageConsumer consumer = session.createConsumer(
    destination,
    "NOT (JMSPriority = 3 + 4 AND Age >= 42)" // selector
);
```

Message selectors are simple expressions given as strings. Its language uses a subset of the SQL92 conditional expression syntax. The main purpose is to filter messages that can be received with a message consumer. Any header or property name can be used in the expression. If the value a property is not present, then its value will be `null`. A selector matches a message only when the value of the selector expression evaluates to true. Because there is no compile-time check on the expression, calling `createConsumer` can throw an `InvalidSelectorException` if the given expression is syntactically incorrect.

Replying to messages

- Messages can contain a ``reply to'' destination
- The value of `JMSReplyTo` header, which can be `null`

- A response message can be linked to the original
  - Using the `JMSCorrelationID` header
  - Often contains the original message's identifier
- Example 11

```
// sending the original message
original.setJMSReplyTo(destination);
producerA.send(original);

// when original message is received
reply.setJMSCorrelationID(received.getJMSMessageID());
producerB.send(received.getJMSReplyTo(), reply);
```

The `JMSReplyTo` and `JMSCorrelationID` headers are providing a simple way to create entire conversations from individual messages. When a message may need a reply, its `JMSReplyTo` field should be filled with a destination. The receiver of this message can extract the information, and send a reply message to the given destination. To link the messages together, the reply message can refer to the original message in its `JMSCorrelationID` header. When it contains a message identifier, it must start with ``ID:''.

Temporary destination

- `TemporaryQueue` and `TemporaryTopic` interfaces
  - Based on `Destination`, `Queue` and `Topic`
  - They just add a `delete()` method to the existing ones
- Created by the application
- Bound to the lifetime of the `Connection`
- Can be consumed only in the producer `Connection`
- Example 12

```
Destination destination = session.createTemporaryQueue();
```

OR

```
Destination destination = session.createTemporaryTopic();
...
destination.delete();
```

If the application needs temporary destinations for internal communication, they can be created using a session object. The created destinations will be unique, and they can be reached only from the connection in which they were created. Because destinations can use resources outside the Java virtual machine, they are heavyweight objects and should be removed with a call to method `delete()` when they are no longer needed. It will throw an exception when there are active consumers on the temporary destination.

`QueueBrowser`

- A queue can be examined without change
- Does not work with topics
- Message selector can be used
- Example 13

```
QueueBrowser browser = session.createBrowser((Queue)destination);
Enumeration<Message> messages = browser.getEnumeration();
...
```

Queue browsers can be used to examine messages in a queue by enumerating them without removing. The order of enumeration will be the same as the order of receiving would be. Queue browsers cannot be used with topics. They should be closed after they are no longer needed.

Message Driven Bean

- Special Enterprise Bean to process `JMS` messages
- Uses the `MessageListener` interface
- Activated when a message is available on the destination
- The connection and session is provided and configured by the container
- Connection factory and destination is container-configured
  - With a configuration file
  - Or using annotations
- Example 16

Simplified `API`

Elements of the simplified `JMS API`

- New interfaces

  - JMSContext
  - JMSProducer, JMSConsumer
- It also uses some elements of the classic API
  - ConnectionFactory, Destination and Message remains the same
  - Derived interfaces of these are also used
- Example 17

JMS 2.0 provides a simplified API to support programming of message oriented applications with less coding effort. It keeps the interfaces related to connection factories, destinations and messages, but integrates the Connection and Session objects into a single JMSContext object.

The API replaces MessageProducer with a new JMSProducer interface which can be used to build and send messages with a single chain of calls. Classical message consumers are also replaced by a simpler JMSConsumer interface, which is able to extract the body of the messages more simpler.

To make the usage more comfortable, this API uses the unchecked JMSRuntimeException to indicate errors, instead of the checked JMSException.

Context

- Integrates Connection and Session
- Has a started/stopped state and a session mode
- Created from a connection factory
- Or injected by the application container

```
JMSContext context = factory.createContext(sessionMode);
```

OR

```
@JMSConnectionFactory("java:global/jms/myConnectionFactory")
@Inject private JMSContext context;
```

OR

```
JMSContext otherContext = context.createContext(sessionMode);
```

Since most of the applications are using a single connection with a single session, the simplified API provides a context object that integrates them. This way its interface combines the connection and session interfaces. While its internal connection is created in a stopped state, it will be automatically started when the first consumer is

created. To disable this functionality, e.g. to set up multiple consumers before starting the connection, the `setAutoStart` method should be called with a `false` parameter before creating the first consumer.

The `createContext` method is overloaded, thus provides various parametrization options. On creation of a context, we can optionally specify all parameters which can be used to construct a connection or session: authorization data for the connection provided as user name and password strings, and session mode for the session.

A context is able to create further contexts which are using the same connection as itself. In this case, the internal connection instance will be shared, and the new context instance will have its own new session object. As every context encapsulates an own session, they must be used only from a single thread. Context objects should be closed when they are no longer needed.

Producer

- `JMSProducer` replaces the `MessageProducer` interface
- Provides a builder interface to prepare and send messages
- Message headers and properties can be set with chained calls of setters
- The `send` method closes the chain and finally sends the assembled message

```
JMSProducer producer = context.createProducer();
producer.setDeliveryMode(deliveryMode) // adjust delivery mode
        .setPriority(7) // set message priority
        .setTimeToLive(42 * 1000) // set expiration to 42 seconds
        .setProperty("Age", 42) // set a custom property
        // it will be a text message (send has many overloads)
        .send(destination, "message body")
        // send with same properties and headers
        .send(destination, "another message");
producer.clearProperties(); // remove all custom properties
...
```

The new `JMSProducer` interface enables to build and send messages with method chaining (a.k.a. method cascading). It maintains an internal state, where every setting for header values like priority, and every custom property values are stored. The user can alter these settings by using setter methods which are returning the same producer object, and this way enable chaining of method calls. When the `send` method is called at the end of the chain, it creates a message object, fills its headers and properties from its internal state. Then it sends the message to the given destination with the given message body. A call to `clearProperties` can be used to remove all custom properties from its internal state, and this way the next messages will not contain any of them.

Consumer

- `JMSConsumer` replaces the `MessageConsumer` interface
- Provides special methods to extract the body of messages
  - Directly by using specialized version of receive
  - Classic receive methods are also available

```
JMSConsumer consumer = context.createConsumer(destination);
// receive a text message with 1 second timeout
// extract and return its body directly when succeeded
String text = consumer.receiveBody(String.class, 1000);
```

OR

```
// non-blocking version
String text = consumer.receiveBodyNoWait(String.class);
```

The new message consumer in the simplified `API` basically works the same way as an ordinary `MessageConsumer`. To enable simpler usage, it offers methods to receive a message and extract its body in a single call. It provides three new methods to achieve this. The first version of `receiveBody` uses a single parameter which determines what kind of payload should be extracted from the message body after receive. The second version of this method provides an additional timeout parameter, which works the same as in classic `API`. A non-blocking version, `receiveBodyNoWait` is also supported.

The type parameter of these methods is depending on the type of the incoming message. For text messages, it must be `String.class`. For object messages `java.io.Serializable.class`, for map messages `java.util.Map.class` or a `java.lang.Object.class`. Finally, for byte messages it must be `byte[].class` or `java.lang.Object.class`. These methods will throw a `MessageFormatRuntimeException` when the extraction of the message body is not succeeded.